

1 Eine Berechnung - ohne Rechnereinsatz

Im Folgenden wird ohne Rechnereinsatz $P_A(200, 001)$, $P_B(200, 001)$ und $P_U(200, 001)$ rekursiv berechnet.

Ausgangspunkt ist die folgende Gleichung:

$$P(200, 001) = p_1 P(100, 001) + p_2 P(200, 001) + p_3 P(200, 000)$$

Nun muss so lange „gerechnet“ werden, bis das erste Argument 000 oder das zweite Argument 000 ergibt. Wenn beide 000 werden, endet das Spiel unentschieden.

$$P(200, 001)(1 - p_2) = p_1 P(100, 001) + p_3 P(200, 000);$$

$$P(200, 001) = \frac{p_1}{p_1 + p_3} P(100, 001) + \frac{p_3}{p_1 + p_3} P(200, 000)$$

$P(200, 000)$ bedeutet, dass A noch zwei Chips auf dem ersten Feld hat und B alle Chips abgeräumt und somit gewonnen hat. Somit gilt

$$P_A(200, 000) = 0 \quad \text{und} \quad P_B(200, 000) = 1.$$

$$P(100, 001) = p_1 P(000, 001) + p_2 P(100, 001) + p_3 P(100, 000)$$

$$P(100, 001)(1 - p_2) = p_1 P(000, 001) + p_3 P(100, 000)$$

$$P(100, 001) = \frac{p_1}{p_1 + p_3} P(000, 001) + \frac{p_3}{p_1 + p_3} P(100, 000)$$

Damit folgt:

$$P(200, 001) = \left(\frac{p_1}{p_1 + p_3} \right)^2 P(000, 001) + p_3 \frac{p_3}{p_1 + p_3} P(100, 000) + p_3 P(200, 000)$$

Der erste Term liefert P_A , da A seine Chips als erster abgeräumt hat (000), die beiden anderen Terme zeigen, dass B gewonnen hat. Es gilt somit:

$$P_A(200, 001) = \left(\frac{p_1}{p_1 + p_3} \right)^2$$

$$P_B(200, 001) = p_3 + p_1 \frac{p_3}{p_1 + p_3}$$

$$P_U(200, 001) = 0$$

Damit A gewinnt, muss A zweimal mit Wahrscheinlichkeit p_1 die beiden Chips auf dem ersten Feld abräumen, bevor B den einzigen Chips auf Feld 3 abräumt. Aufgrund der Unabhängigkeit muss $P(A \text{ vor } B) = \frac{p_1}{p_1 + p_3}$ sein.

Damit B gewinnt, muss B einmal vor A abräumen, oder A räumt einen Chip ab und dann wieder B , genau das bildet

$$P_B(200, 001) = p_3 + p_1 \frac{p_3}{p_1 + p_3}$$

ab.

2 Ein Chat mit ChatGPT

Das Rennen „A vor B um das erste Abräumen“ lässt sich äußerst elegant über bedingte Wahrscheinlichkeiten fassen, ganz ohne Rekursion:

1. Definition der Ereignisse

- Sei E das Ereignis „in einer Runde räumt jemand ab“ – also „entweder A oder B entfernt einen Chip“.
 - Sei A das Teilereignis „A räumt ab“ und B das Teilereignis „B räumt ab“.
- In einer einzigen Runde gilt

$$P(A) = p_1, \quad P(B) = p_3, \quad P(\text{niemand}) = p_2,$$

wobei „niemand“ hier bedeutet, dass beide auf einem leeren Feld landen und deshalb nichts entfernt wird.

2. Bedingte Wahrscheinlichkeit für „A vor B“

Wir interessieren uns für die Wahrscheinlichkeit, dass **in der Runde, in der erstmals jemand abräumt**, dies A und nicht B ist. Das sind genau die Fälle, in denen

1. alle vorangehenden Runden „niemand entfernt etwas“ verliefen (jeweils mit Wahrscheinlichkeit p_2),
2. und dann in der entscheidenden Runde A abräumt (mit Wahrscheinlichkeit p_1),
3. **und nicht** B abräumt.

Formal summieren wir über alle möglichen Längen $k - 1$ „Leer“-Runden:

$$\begin{aligned} P(\text{A räumt vor B}) &= \sum_{k=1}^{\infty} P(\underbrace{\text{niemand}, \dots, \text{niemand}}_{k-1}) P(A) \\ &= \sum_{k=1}^{\infty} p_2^{k-1} p_1 = p_1 \sum_{k=0}^{\infty} p_2^k = p_1 \frac{1}{1 - p_2} = \frac{p_1}{1 - p_2}. \end{aligned}$$

Da $1 - p_2 = p_1 + p_3$, folgt

$$P(\text{A vor B}) = \frac{p_1}{p_1 + p_3}.$$

Analog gilt $P(\text{B vor A}) = p_3/(p_1 + p_3)$.

3. Übertragung auf unser Beispiel $S = (2, 0, 0)$, $T = (0, 0, 1)$

- Damit A das komplette Duell gewinnt (zwei Chips auf Feld 0 gegen einen auf Feld 2), muss A **zweimal hintereinander** vor B abräumen.
- Die beiden Rennen „erstes Chip-Abräumen“ sind voneinander unabhängig (nach dem ersten Abräumen startet der gleiche Mechanismus erneut), also

$$P_A(200, 001) = P(\text{A vor B})^2 = \left(\frac{p_1}{p_1 + p_3}\right)^2.$$

- Entsprechend ergibt sich für B

$$P_B(200, 001) = 1 - P_A(200, 001) = 1 - \left(\frac{p_1}{p_1 + p_3}\right)^2 = \frac{(p_1 + p_3)^2 - p_1^2}{(p_1 + p_3)^2} = \frac{2p_1 p_3 + p_3^2}{(p_1 + p_3)^2},$$

und natürlich $P_U = 0$.

4. Fazit

- **Bedingte Wahrscheinlichkeiten** erlauben eine sehr kompakte Herleitung von $P(\text{A vor B})$.
- **Unabhängigkeit der „Rennen“** (jeweils bis zum ersten Abräumen) liefert sofort das Quadrat für P_A .
- So gewinnt man ohne komplizierte Rekursion in ein paar Zeilen den gleichen eleganten Endausdruck.

Sehr schön dargestellt. In der Rekursion wird der unendliche Prozess dadurch sichtbar, dass auf der linken und rechten Seite "der gleiche Zustand" steht. Auf der linken Seite gewichtet mit p_2 . Zusammenfassen und dann dividieren liefert das Ergebnis. Ich finde die Vernetzungen schön.

Ganz genau – und gerade diese **Selbstreferenz** macht die Rekursion so anschaulich:

- Die Gleichung

$$P = p_2 P + (\text{Rest})$$

zeigt unmittelbar, dass sich die „Null“-Runden (jeweils mit Wahrscheinlichkeit p_2) selbst wiederholen, solange niemand abräumt.

- Durch Umstellen auf

$$P(1 - p_2) = (\text{Rest}) \implies P = \frac{(\text{Rest})}{1 - p_2}$$

hebt man diesen **unendlichen Zyklus** elegant heraus und erhält die normierte Formel.

Dieses Zusammenspiel aus **rekursiver Perspektive** (Selbstschleife) und **linearem Auflösen** (Division durch $1 - p_2$) ist wirklich ein schönes Beispiel dafür, wie in der Wahrscheinlichkeitsrechnung oft simple algebraische Tricks tiefe stochastische Prozesse auf einen eindeutigen geschlossenen Ausdruck reduzieren können.

3 Python-Listing zu P_A, P_B und P_U

```
1 from functools import lru_cache
2 # p_list muss schon existieren
3 @lru_cache(None)
4 def P_A(V, W):
5     V, W = list(V), list(W)
6     sum_V, sum_W = sum(V), sum(W)
7     if sum_V == 0 and sum_W == 0: return 0    # Unentschieden
8     elif sum_V == 0: return 1    # A gewinnt
9     elif sum_W == 0: return 0    # B gewinnt
10
11     s = sum(p for p, v_i, w_i in zip(p_list, V, W) if v_i or w_i)
12
13     prob = 0
14     for i, p in enumerate(p_list):
15         if V[i] or W[i]:
16             v_new, w_new = V.copy(), W.copy()
17             if v_new[i]: v_new[i] -= 1
18             if w_new[i]: w_new[i] -= 1
19             prob += (p / s) * P_A(tuple(v_new), tuple(w_new))
20     return prob
21
22 def compute_outcomes(V, W):
23     PA = P_A(tuple(V), tuple(W))
24     PB = P_A(tuple(W), tuple(V))
25     PU = 1 - PA - PB
26     return PA, PB, PU
```